TOPIC 02

Linear Regression and Simple Linear Regression

What linear regression is, its types, and a step-by-step, hand-checked guide to simple linear regression

WHAT YOU WILL LEARN

- Explain what **regression** is and what **linear** really means in *linear regression*
- Name the **types of linear regression** (Simple Linear, Multiple Linear, Polynomial, Ridge, Lasso and Elastic Net Regression) and say when each is used
- Write the **simple linear regression** equation and explain the slope, intercept and error term
- Explain **least squares**: why the best line minimises the sum of squared errors
- **Compute a regression line by hand** and check it against scikit-learn
- Measure fit with **SSE, SST, SSR, R^2, MSE, RMSE and MAE**
- Build, evaluate and interpret a model on a real salary dataset, and read its **regression table**
- Check the **assumptions**, and avoid the usual **mistakes** (extrapolation, outliers, no-intercept fits)

Part A. Linear regression and its types

1. What is regression?

In the introduction to machine learning you met the two big kinds of supervised learning. **Regression** predicts a **number** on a continuous scale (a salary, a price, a temperature). **Classification** predicts a **category** (spam or not spam, disease or no disease). The data always has an independent variable X to predict *from* and a dependent variable Y to predict.

	Regression	Classification
Y is	A number that can take many values	A label from a fixed set
Examples	Salary, house price, rainfall, demand	Yes/No, fraud/genuine, cat/dog/bird
Typical metrics	MAE, MSE, RMSE, R^2	Accuracy, precision, recall, F1
Simplest model	Linear regression	Logistic regression

DEFINITION **Linear regression**

A statistical and machine-learning method that **models the relationship between a dependent variable (Y) and one or more independent variables (X) by fitting a linear equation to the observed data**. The best-fitting line is the one that minimises the sum of the squared differences between the observed and the predicted values.

> **NOTE** **A name to be careful with**
> *Logistic* regression has 'regression' in its name, but it is a **classification** method. It is not part of the linear regression family covered here.

2. What does *linear* mean?

'Linear' does **not** mean that the picture must be a straight line. It means the prediction is built by **multiplying each input by a number (a coefficient) and adding the results**. The model is linear in its *coefficients*.

Model	Formula	Linear regression?
Straight line	$y = b_0 + b_1x$	Yes
Several inputs	$y = b_0 + b_1x_1 + b_2x_2$	Yes
Curve using x^2	$y = b_0 + b_1x + b_2x^2$	Yes: still linear in b_0, b_1, b_2 (treat x and x^2 as two inputs)
Exponential	$y = b_0 \cdot e^{(b_1x)}$	No, the coefficient b_1 sits inside the exponent (although taking logs can sometimes fix this)

3. Types of linear regression

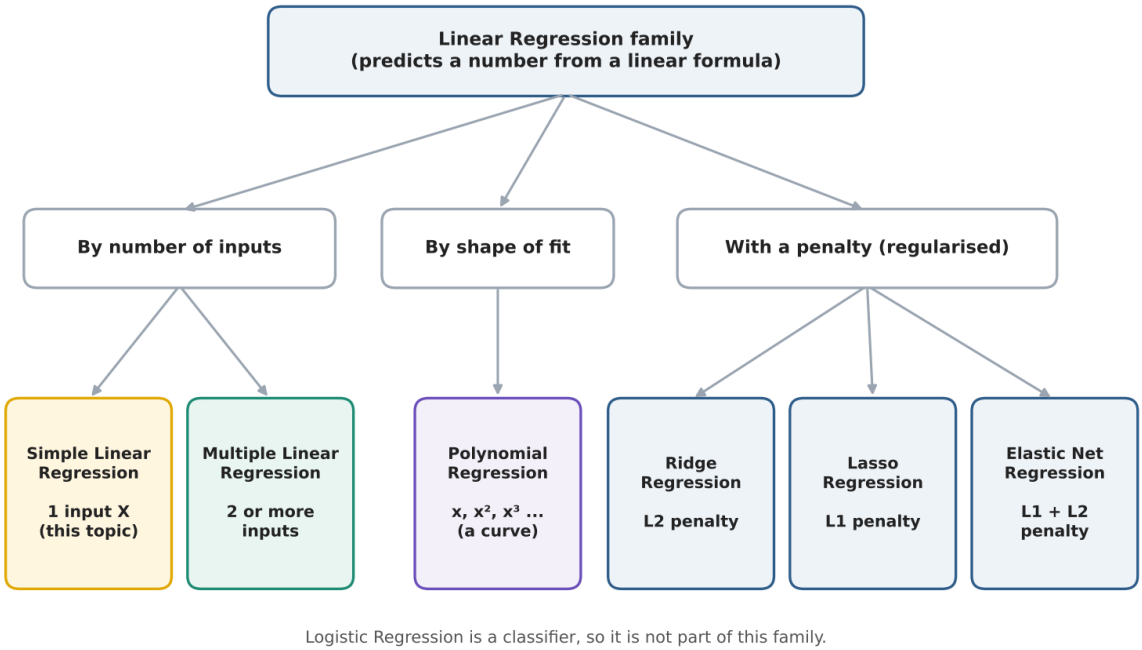


Figure 1. The linear regression family and the full name of each model. This topic goes deep on the first one: Simple Linear Regression.

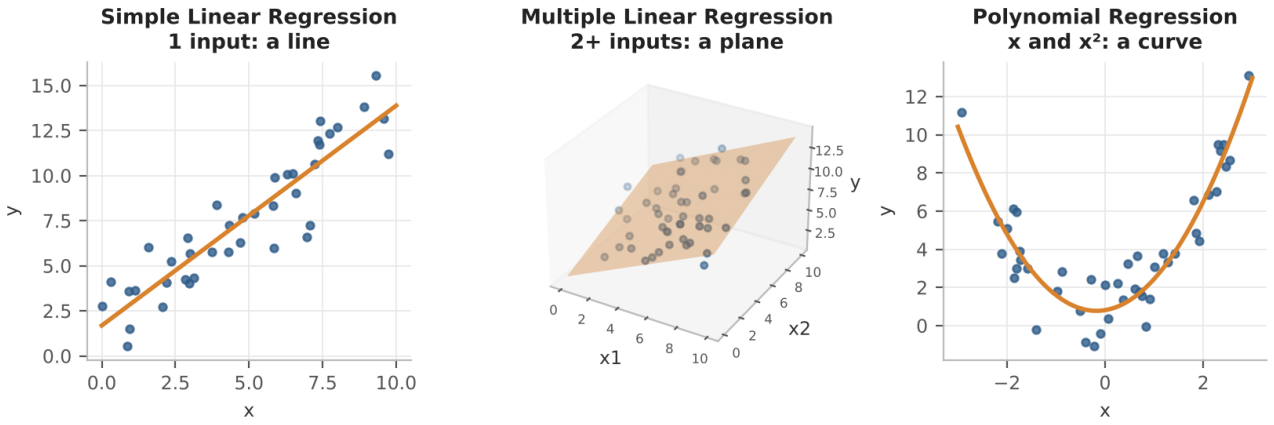


Figure 2. Simple Linear Regression fits a line, Multiple Linear Regression fits a plane (or a higher-dimensional flat surface), Polynomial Regression fits a curve.

The first three models are fitted by **ordinary least squares (OLS)**. The last three add a **penalty** that keeps the coefficients small, which is called **regularisation**, so together they are known as **regularised linear regression**.

Model	Equation	Inputs	Use it when	scikit-learn
Simple Linear Regression (SLR)	$\hat{y} = b_0 + b_1x$	1	One input explains the output; you want an easy, interpretable model	`LinearRegression`
Multiple Linear Regression (MLR)	$\hat{y} = b_0 + b_1x_1 + b_2x_2 + \ldots$	2 or more	Several factors drive the output (price from area, rooms, age)	`LinearRegression`
Polynomial Regression	$\hat{y} = b_0 + b_1x + b_2x^2 + \ldots$	1 (as powers)	The scatter bends but one smooth curve fits	`PolynomialFeatures` + `LinearRegression`
Ridge Regression (L2 regularisation)	Least squares + penalty on the size of b (L2)	Many	Inputs are correlated, or the model overfits: shrinks coefficients	`Ridge`
Lasso Regression (L1 regularisation)	Least squares + penalty on $\lvert b \rvert$ (L1)	Many	You want to drop useless inputs: some coefficients become exactly 0	`Lasso`
Elastic Net Regression (L1 + L2 regularisation)	Least squares + a mix of the L1 and L2 penalties	Many	Many correlated inputs and you also want selection	`ElasticNet`

KEY IDEA **How the types relate**

Every type above is a **generalisation of simple linear regression**: more inputs (Multiple Linear Regression), transformed inputs (Polynomial Regression), or a penalty to keep coefficients small (Ridge, Lasso and Elastic Net Regression). If you fully understand the simple version (equation, least squares, residuals, R^2, assumptions) the others are extensions, not new ideas. That is why the rest of this topic goes deep on it.

NOTE **Multiple versus multivariate**

Multiple linear regression has several inputs and **one** output. **Multivariate** regression has several **outputs** predicted together. Beginners often mix the two terms up.

Part B. Simple linear regression

4. The model

DEFINITION **Simple linear regression (SLR)**

Linear regression with **exactly one independent variable**. It describes the relationship between X and Y with a **straight line**, and uses the line to predict Y for a new X.

$$Y = \beta_0 + \beta_1 X + \varepsilon \qquad \hat{y} = b_0 + b_1 x$$

Symbol	Name	Meaning	Other names
Y (or y)	Dependent variable	What we predict	Target, response, label
X (or x)	Independent variable	What we predict from	Feature, predictor, regressor
β_0 / b_0	Intercept	Predicted y when x = 0 (where the line crosses the y axis)	c in y = mx + c; `intercept_` in scikit-learn
β_1 / b_1	Slope	Predicted **change in y for each 1-unit increase in x**	m in y = mx + c; `coef_` in scikit-learn
ε	Error term	The part of y the line cannot explain (chance, other factors)	Disturbance, noise
$\hat{y}$	Predicted value	The y the fitted line gives for a given x	Fitted value, prediction
$e = y - \hat{y}$	Residual	Actual minus predicted: how far off one prediction is	Error (of one row)

KEY IDEA **Greek versus Latin letters**

β_0 and β_1 are the **true, unknown** population values. b_0 and b_1 are the **estimates** we compute from a sample. Your class notes write the same line as y = mx + c, so m = b_1 (slope) and c = b_0 (intercept). All three notations mean the same thing.

4.1 Reading the slope and the intercept

For the salary data used later in this topic, the fitted line is **Salary = 25,792.20 + 9,449.96 × YearsExperience**.

- **Slope 9,449.96:** on average, each extra year of experience goes with a salary about 9,450 higher.
- **Intercept 25,792.20:** the model's prediction for 0 years of experience. The data only contain people with 1.1 to 10.5 years, so this number is a **mathematical anchor of the line**, not a reliable claim about beginners.

COMMON MISTAKE **Slope is not cause**

A slope says that X and Y *move together* in the data. It does not prove that changing X would change Y. Experience may raise pay, but other things that come with experience (seniority, role, skills) are tangled up in that slope.

5. When is simple linear regression the right tool?

1. Y is a **number** and so is X (categorical inputs need encoding first).
2. You have a clear reason to expect Y to depend on X (the question makes sense).
3. A **scatter plot** of Y against X looks roughly like a **straight band**, not a curve or a cloud.
4. The correlation is not zero. Check `dataset.corr()` first; a value near ±1 means a strong line.
5. You do not extrapolate far outside the X range you trained on.

Always draw the scatter plot **before** fitting anything. Section 11 shows four datasets that have the *same* fitted line and *different* stories.

6. How the best line is found: least squares

6.1 Residuals

Many lines could be drawn through a cloud of points. To choose one we need a score for 'how well does this line fit?'. The **residual** of each point is the vertical gap between the actual y and the line's prediction ŷ.

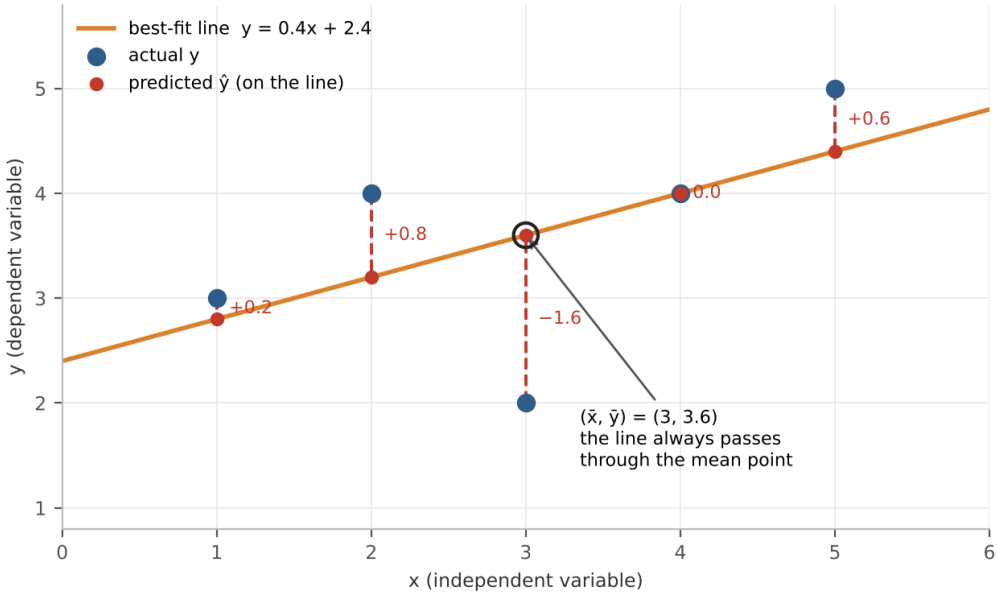


Figure 3. The five-point example from the class slides. Dashed red lines are the residuals (actual minus predicted).

6.2 Why square the residuals?

Adding the raw residuals does not work: positives and negatives cancel, so a terrible line can total zero. Squaring makes every error positive, and punishes big misses much more than small ones. The score to minimise is the **sum of squared errors**:

$$SSE = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{n} (y_i - b_0 - b_1 x_i)^2$$

The method of choosing b_0 and b_1 to make SSE as small as possible is called **ordinary least squares (OLS)**. For the five-point example, SSE is smallest for one particular slope and no other:

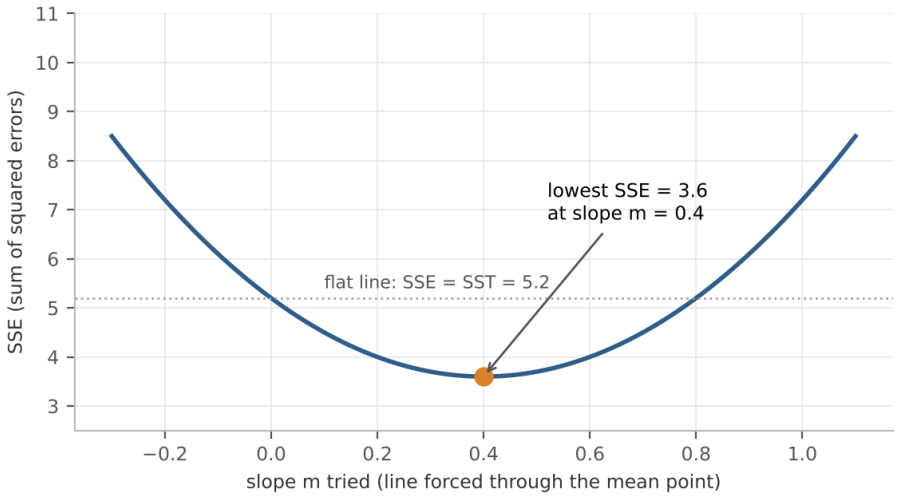


Figure 4. SSE as a function of the slope (for lines through the mean point). It is a smooth bowl with exactly one lowest point, m = 0.4, so the best line is unique.

6.3 The formulas

Setting the derivatives of SSE with respect to b_0 and b_1 to zero gives a closed-form answer. No guessing or searching is needed:

$$b_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2} \qquad b_0 = \bar{y} - b_1\bar{x}$$

Piece	Meaning
$\Sigma(x - \bar{x})(y - \bar{y})$	How x and y **vary together** (covariance, before dividing by n−1). Positive when high x goes with high y.
$\Sigma(x - \bar{x})^2$	How much x **varies by itself** (variance, before dividing by n−1).
b_1 = together ÷ alone	The slope is the joint variation of x and y compared with x's own variation.
$b_0 = \bar{y} - b_1\bar{x}$	Rearranging $\hat{y} = b_0 + b_1x$ at the point $(\bar{x}, \bar{y})$: the line is forced to pass through the **mean point**.

KEY IDEA **An equivalent form**

The slope can also be written $b_1 = r \cdot (s_y / s_x)$, where r is the correlation and s_y, s_x are the standard deviations. It shows that the slope has the **same sign as the correlation** and depends on the **units** of x and y.

7. Worked example by hand

This is the example from the class slides. Five observations: x = 1, 2, 3, 4, 5 and y = 3, 4, 2, 4, 5. Follow each step and check the numbers; they are the same ones the machine uses.

1. **Means.** $\bar{x} = (1+2+3+4+5)/5 = \mathbf{3}$; $\bar{y} = (3+4+2+4+5)/5 = \mathbf{3.6}$.
2. **Deviations** from the means, for every row: $x - \bar{x}$ and $y - \bar{y}$.
3. **Multiply** the deviations $(x - \bar{x})(y - \bar{y})$ and **square** the x deviations $(x - \bar{x})^2$.
4. **Add up** the two columns, then divide: $b_1 = \Sigma(\text{product}) \div \Sigma(\text{square})$.
5. **Intercept:** $b_0 = \bar{y} - b_1\bar{x}$.
6. **Predict**, then compute the residuals and SSE.

x	y	$x - \bar{x}$	$y - \bar{y}$	$(x - \bar{x})^2$	$(x - \bar{x})(y - \bar{y})$
1	3	−2	−0.6	4	1.2
2	4	−1	0.4	1	−0.4
3	2	0	−1.6	0	0.0
4	4	1	0.4	1	0.4
5	5	2	1.4	4	2.8
3	**3.6**			**Σ = 10**	**Σ = 4**

$$b_1 = \frac{4}{10} = 0.4 \qquad b_0 = 3.6 - 0.4 \times 3 = 2.4 \qquad \hat{y} = 0.4x + 2.4$$

x	y (actual)	$\hat{y} = 0.4x + 2.4$	residual $y - \hat{y}$	residual²
1	3	2.8	0.2	0.04
2	4	3.2	0.8	0.64
3	2	3.6	−1.6	2.56
4	4	4.0	0.0	0.00
5	5	4.4	0.6	0.36
			Sum = 0	**SSE = 3.6**

KEY IDEA **Two quick self-checks**

(1) The residuals of a least-squares line **always add up to zero** (here 0.2 + 0.8 − 1.6 + 0 + 0.6 = 0). (2) The line passes through the mean point (3, 3.6): 0.4 × 3 + 2.4 = 3.6. If either check fails, there is an arithmetic slip.

From SSE we get the error measures: **MSE** = 3.6 ÷ 5 = 0.72, **RMSE** = √0.72 ≈ 0.85, **MAE** = (0.2 + 0.8 + 1.6 + 0 + 0.6) ÷ 5 = 0.64. Section 8 explains what they mean.

Same answer from scikit-learn

```
import numpy as np
from sklearn.linear_model import LinearRegression

x = np.array([[1], [2], [3], [4], [5]])      # X must be 2-D: rows x columns
y = np.array([3, 4, 2, 4, 5])
model = LinearRegression().fit(x, y)
print(model.coef_[0], model.intercept_)       # 0.4  2.4
```

8. Measuring how good the line is

8.1 Splitting the variation: SST, SSR, SSE

The total variation of y around its mean splits into a part the line **explains** and a part it **leaves unexplained**:

$$SST = SSR + SSE$$

Term	Formula	Meaning
SST (total)	$\Sigma(y - \bar{y})^2$	All the variation of y around its mean
SSR (explained)	$\Sigma(\hat{y} - \bar{y})^2$	The part the line accounts for
SSE (unexplained)	$\Sigma(y - \hat{y})^2$	The part left over: the sum of squared residuals

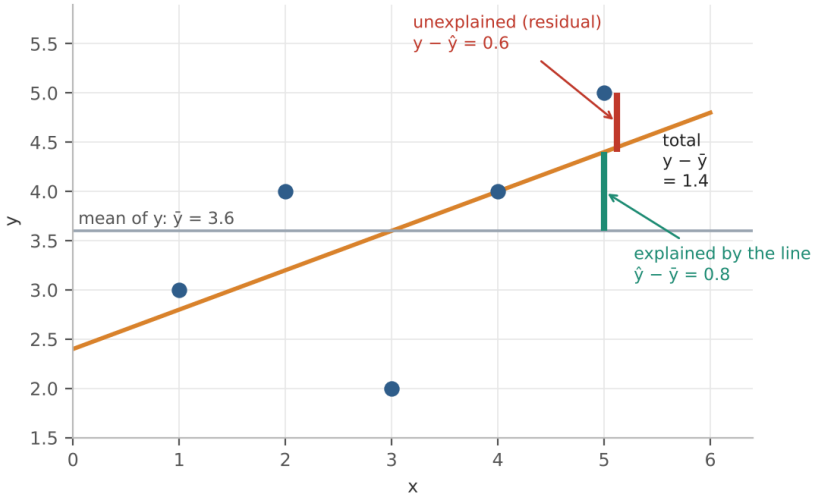


Figure 5. For the point x = 5: its total distance from the mean (1.4) is the explained part (0.8) plus the residual (0.6). Summing the squares over all points gives SST, SSR and SSE.

In the example: SST = 5.2, SSR = 1.6, SSE = 3.6, and indeed 1.6 + 3.6 = 5.2.

8.2 R², the coefficient of determination

$$R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$$

R² is the **share of the variation in y that the model explains**. It runs from 0 (no better than always predicting ȳ) to 1 (perfect). For the example, R² = 1 − 3.6/5.2 = **0.308**: the line explains about 31% of the variation, which fits the loose scatter you see in Figure 3.

> **KEY IDEA** **In simple regression, R² = r²**
> With one input, R² is the **square of the correlation** between x and y. Example: r = 0.5547, so r² = 0.3077 = R². For the salary data r = 0.9782 and R² = 0.9570.

8.3 Error measures in the units of y

Measure	Formula	Meaning	Five-point example
MAE	mean of $\lvert y - \hat{y} \rvert$	Typical size of a miss, in y's units. Not dominated by outliers.	0.64
MSE	mean of $(y - \hat{y})^2$	Average squared miss. Penalises large misses heavily; units are y².	0.72
RMSE	√MSE	Typical miss **in y's own units**; the most reported error.	0.85
R²	1 − SSE/SST	Share of the variation explained (no units).	0.308

COMMON MISTAKE **A high R^2 is not a guarantee**

R^2 measured on the data the line was fitted to always flatters it. Judge a model on **test data** it has not seen, and look at the residual plots (Section 11). A line can have a high R^2 and still be the wrong shape, as the Anscombe datasets show.

9. A real example: salary and experience

9.1 The data

`Salary_Data.csv` has **30 rows and 2 columns** and no missing values. `YearsExperience` (X) runs from 1.1 to 10.5 years and `Salary` (Y) from 37,731 to 122,391. The mean experience is 5.31 years and the mean salary is 76,003. The correlation between them is **r = 0.978**, so a straight line is a sensible first model.

9.2 The code, step by step

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

dataset = pd.read_csv("Salary_Data.csv")
x = dataset.iloc[:, :-1]          # independent variable: a 2-D DataFrame
y = dataset.iloc[:, -1]           # dependent variable: a 1-D Series

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=0)      # 24 rows train, 6 rows test

regressor = LinearRegression()
regressor.fit(x_train, y_train)               # learn b0 and b1 (training)
y_pred = regressor.predict(x_test)            # predict the unseen rows

print(regressor.intercept_, regressor.coef_)  # b0, b1
print(regressor.score(x_train, y_train))      # R^2 on training data
print(regressor.score(x_test, y_test))        # R^2 on test data
```

Line	What it does and why
`iloc[:, :−1]`	Every column except the last is X. Keeping X as a DataFrame keeps it **2-D (rows × columns)**, which scikit-learn requires, even with one column.
`iloc[:, −1]`	The last column is Y. Y is 1-D.
`train_test_split(..., test_size=0.2, random_state=0)`	20% of rows are set aside as unseen test data. The fixed `random_state` makes the split repeatable.
`.fit(x_train, y_train)`	**Training**: least squares computes b_0 and b_1 from the training rows only.
`.predict(x_test)`	Applies $\hat{y} = b_0 + b_1 x$ to the test rows.
`.score(x, y)`	Returns R^2 for the data given.

COMMON MISTAKE **Common error: 'Expected 2D array, got 1D array'**

This appears if X is a 1-D array or Series. Fix it with `x.values.reshape(−1, 1)` (one column, as many rows as needed) or by selecting `dataset[['YearsExperience']]` with **double** brackets.

9.3 Results on the test rows

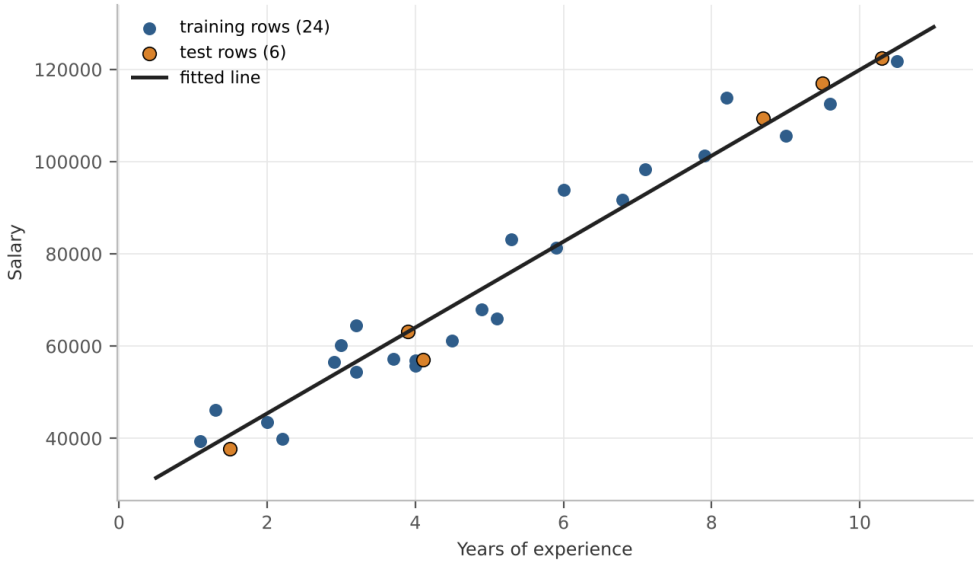


Figure 6. The line is learned from the 24 blue training points; the 6 orange test points were never used to fit it.

Quantity	Value	Meaning
Intercept b_0	26,780.10	Predicted salary at 0 years (anchor of the line)
Slope b_1	9,312.58	Salary rises by about 9,313 per extra year
R^2 on training rows	0.941	Fit to the rows used for learning
R^2 on test rows	0.988	Fit to rows the model has never seen
RMSE on test rows	3,581	Typical miss on unseen rows, in salary units
MAE on test rows	2,446	Average absolute miss

YearsExperience	Actual salary	Predicted salary	Residual
1.5	37,731	40,749	−3,018
10.3	122,391	122,700	−309
4.1	57,081	64,962	−7,881
3.9	63,218	63,099	119
9.5	116,969	115,250	1,719
8.7	109,431	107,800	1,631

COMMON MISTAKE **Do not trust one small test split**

Only 6 rows are in the test set, so its R^2 depends heavily on which rows landed there. Repeating the split with 50 different random seeds gave test R^2 values from **0.76 to 0.99** (median 0.94). The single value 0.988 above is toward the lucky end. For small datasets prefer **cross-validation**, and quote a typical value, not one split.

After you are satisfied with the model, it is usual to refit on **all 30 rows** before using it. That final line is **Salary = 25,792.20 + 9,449.96 × Years** with $R^2 = 0.957$.

9.4 Predicting new values, and why 20 years is risky

Using the line fitted on the training rows, the predicted salary for 12 years is **138,531** and for 20 years is **213,032**. Both are **extrapolations**: the data stop at 10.5 years.

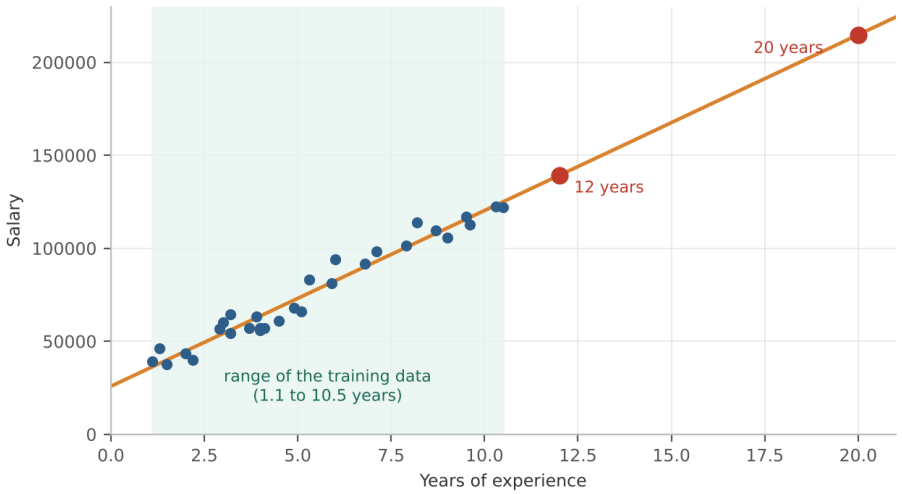


Figure 7. Inside the green band the line is backed by data. Outside it the line is only an assumption that the same straight trend continues.

> **COMMON MISTAKE** **Extrapolation**
>
> Real salaries level off or jump with role changes, so a straight line is unlikely to stay valid at 20 years. Predictions are trustworthy only **inside the range of X seen in training**. The further outside, the weaker the evidence.

10. The statistical view: the regression table

scikit-learn gives you the line and the scores. **statsmodels** additionally gives the **standard errors, t-tests, p-values and confidence intervals** that tell you whether the slope is real or just sampling noise. This is the 'regression table' from your statistics notes (08_Statistics/03).

```
import statsmodels.api as sm

X = sm.add_constant(x)           # adds the column of 1s for the intercept
model = sm.OLS(y, X).fit()
print(model.summary())
```

Quantity	Value	How to read it
Slope b_1 (std. error)	9,449.96 (378.75)	The standard error says how much b_1 would wobble from sample to sample
t statistic for b_1	24.95	b_1 divided by its standard error; here far beyond ±2
p-value for b_1	< 0.001	Under H_0: $\beta_1 = 0$ (no relationship) a slope this big is essentially impossible, so we reject H_0
95% CI for β_1	8,674 to 10,226	The plausible range for the true slope per extra year
Intercept b_0 (std. error)	25,792.20 (2,273.05)	t = 11.35; 95% CI 21,136 to 30,448
R^2 / adjusted R^2	0.957 / 0.955	Adjusted R^2 penalises extra inputs; with one input it is almost the same
F statistic	622.5	Tests the whole model. In SLR, $F = t^2$ ($24.95^2 = 622.5$)
Residual std. error	5,788	Typical size of the error term ε: $\sqrt{(SSE \div (n - 2))}$

COMMON MISTAKE **Forgetting the constant in statsmodels**

Unlike scikit-learn, `sm.OLS(y, x)` does **not** add an intercept. Without `sm.add_constant`, it fits a line forced through the origin (0, 0). On this data that gives a slope of **13,255** instead of **9,450**, and its R^2 (0.973) is calculated differently and cannot be compared with the normal R^2 (0.957). Always add the constant unless you have a strong reason not to.

11. Assumptions of simple linear regression

The line itself can always be computed. The **tests, confidence intervals and trustworthy predictions** rely on four assumptions about the errors, remembered as **LINE**:

	Assumption	Meaning	How to check	If it fails
L	**Linearity**	The true relationship is a straight line	Scatter plot; residuals vs fitted show no curve	Transform x or y, or use polynomial regression
I	**Independence**	Residuals are not related to each other (no pattern over time or order)	Know how the data was collected; plot residuals in order	Time-series methods; collect independent samples
N	**Normality**	Residuals are roughly bell-shaped (matters mainly for tests and intervals, for small samples)	Histogram and Q-Q plot of residuals	Transform y; use more data; robust methods
E	**Equal variance**	The spread of residuals is the same for all x (homoscedasticity)	Residuals vs fitted: no funnel shape	Transform y (e.g. log); weighted least squares

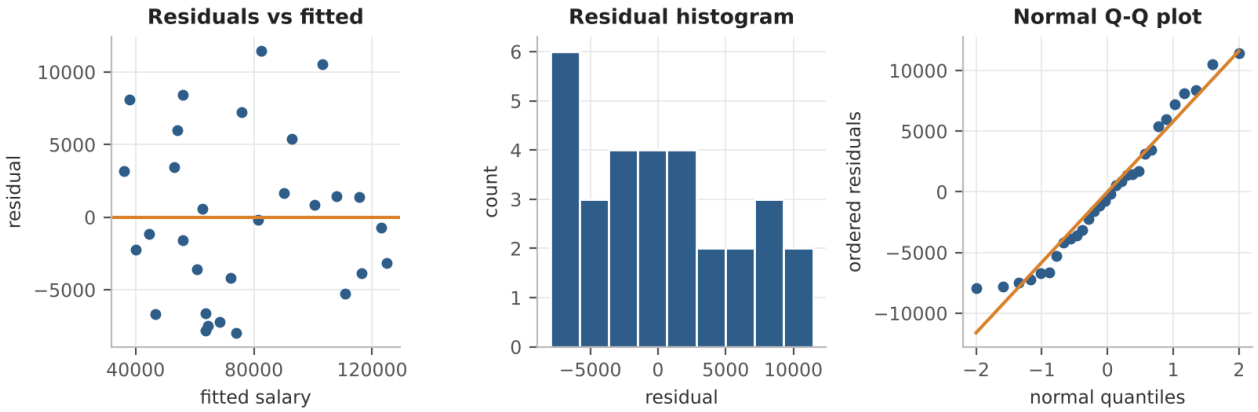


Figure 8. Diagnostics for the salary model. Left: residuals scatter randomly around zero with no curve or funnel. Middle and right: roughly bell-shaped, points close to the Q-Q line.

For the salary model all three plots look healthy. A Shapiro-Wilk test on the residuals gives p = 0.20, so there is no evidence against normality (with only 30 rows, the test has little power, so rely on the plots as well).

11.1 Always plot: Anscombe's quartet

Statistician Francis Anscombe built four small datasets in 1973 that share **almost identical numbers** (same means, same fitted line y = 3.0 + 0.5x, same correlation 0.82 and R^2) yet look completely different. Only the plot reveals the truth.

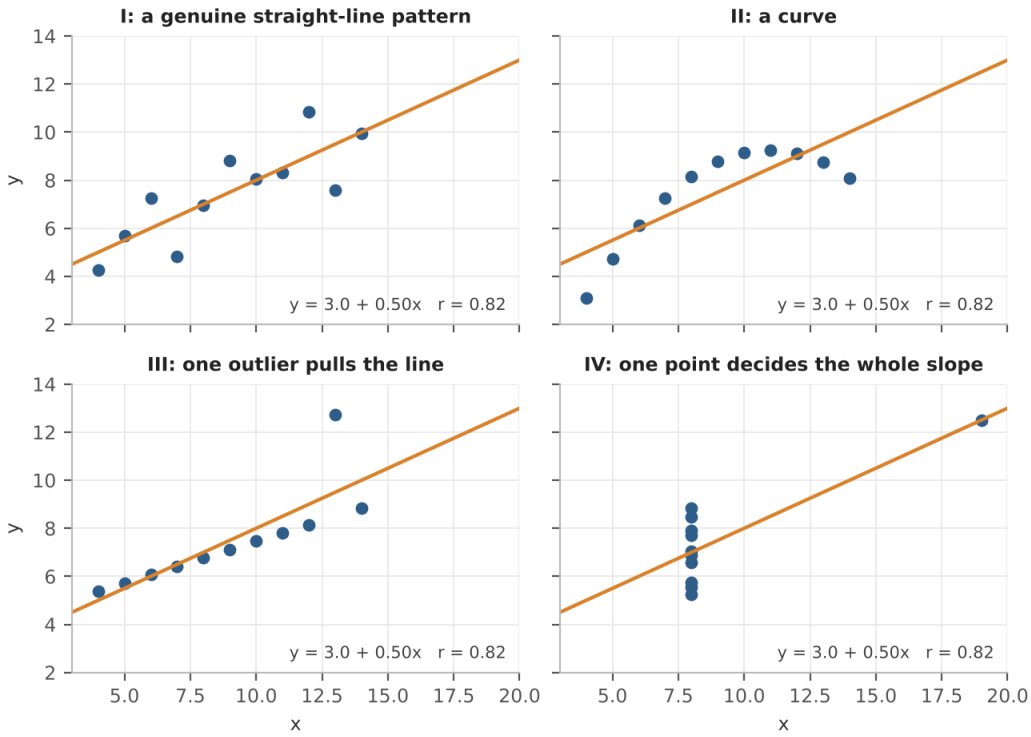


Figure 9. Anscombe's quartet. Only panel I is a good case for a straight line; II needs a curve; III has an outlier; IV is decided by a single point.

KEY IDEA **Lesson**

Summary numbers (slope, r, R^2) can be identical for datasets that need completely different treatment. **Plot the data and the residuals every time.**

12. Two ways to find the line

	Closed-form least squares	Gradient descent
Idea	Use the formulas of Section 6.3 (or the matrix equivalent) to jump straight to the answer	Start with any b_0, b_1, then repeatedly step downhill on the SSE bowl (Figure 4)
Used by	`LinearRegression` in scikit-learn (an ordinary least squares solver), and statsmodels OLS	Many other ML models, such as neural networks, where no formula exists
Result	Exact answer in one calculation	Approaches the same answer after many small steps (needs a learning rate)

The code below runs gradient descent on the five-point example. After enough steps it lands on the same line the formulas gave, which is a nice way to see that 'training' is just 'minimising the loss'.

```
import numpy as np
x = np.array([1, 2, 3, 4, 5.]);  y = np.array([3, 4, 2, 4, 5.])
m, c, lr = 0.0, 0.0, 0.01                  # start at a flat line; lr = learning rate
for _ in range(20000):
    error = (m * x + c) - y                # prediction minus actual
    m -= lr * 2 * (error * x).mean()       # slope of the loss with respect to m
    c -= lr * 2 * error.mean()             # slope of the loss with respect to c
print(round(m, 4), round(c, 4))            # 0.4  2.4
```

13. Common mistakes

Mistake	Why it is a problem	What to do
Fitting without plotting	A straight line may be the wrong shape (Anscombe)	Scatter plot first, residual plots after
Extrapolating far outside the data	The straight-line trend may not continue	Stay inside the training range of X, or collect data there
Reading the slope as cause and effect	Correlation is not causation; hidden factors	Say 'associated with', unless the data come from an experiment
Trusting R^2 alone	Flattering on training data; blind to shape and outliers	Check test-set error and residual plots
Judging on one tiny test split	The score changes a lot between splits	Cross-validation, or report a range
1-D X in scikit-learn	Raises the 'Expected 2D array' error	Keep X two-dimensional
No constant in statsmodels	Fits a line through the origin; different slope and R^2	Use `sm.add_constant(X)`
Comparing slopes across different units	The slope depends on the units of x and y	Compare standardised slopes or r, or state units
Ignoring outliers	One extreme point can tilt the whole line	Investigate each outlier before removing or keeping it

SUMMARY Summary

- Regression predicts a **number**. *Linear* regression builds the prediction from coefficients times inputs, plus a constant.
- Types: **Simple Linear Regression** (1 input), **Multiple Linear Regression** (2+ inputs), **Polynomial Regression** (powers of x), and the regularised **Ridge, Lasso and Elastic Net Regression**. All extend the simple model.
- SLR: $\hat{y} = b_0 + b_1 x$. The slope is the change in y per unit of x; the intercept is the prediction at x = 0.
- **Least squares** picks the line with the smallest SSE: $b_1 = \Sigma(x-\bar{x})(y-\bar{y}) \div \Sigma(x-\bar{x})^2$ and $b_0 = \bar{y} - b_1\bar{x}$. The line passes through $(\bar{x}, \bar{y})$ and its residuals sum to 0.
- **SST = SSR + SSE**; $R^2 = 1 - SSE/SST$ and, in SLR, equals r^2. RMSE and MAE give the error in y's units.
- In scikit-learn: keep X 2-D, `fit` on training rows, `predict` and `score` on test rows. In statsmodels: remember `add_constant`.
- Trust the model only inside the training range of X, and check the **LINE** assumptions with plots. **Always plot the data.**

Practice questions

Try each question before reading the answer underneath it.

Q1. Find the regression line through (1, 2), (2, 4), (3, 5), and predict y at x = 4.

Answer. $\bar{x} = 2$ and $\bar{y} = 11/3 \approx 3.667$. $\Sigma(x-\bar{x})(y-\bar{y}) = (-1)(-1.667) + 0 + (1)(1.333) = 3$ and $\Sigma(x-\bar{x})^2 = 2$, so $b_1 = 1.5$. $b_0 = 3.667 - 1.5 \times 2 \approx 0.667$. The line is $\hat{y} = 0.667 + 1.5x$, giving $\hat{y}(4) \approx 6.67$. (This is a mild extrapolation, since x = 4 is just outside the data.)

Q2. In the salary model the slope is 9,449.96. Explain it in one sentence.

Answer. On average, one extra year of experience goes with a salary about 9,450 higher, across the range of experience in the data. It describes an association, not proof of cause.

Q3. The salary model has $R^2 = 0.957$. What does that mean, and what does it not tell you?

Answer. About 95.7% of the variation in salary is explained by experience. It does not tell you that the model is the right shape, that it will predict new rows equally well, or that experience causes the salary.

Q4. The correlation between experience and salary is 0.9782. What is the R^2 of the simple regression, without fitting anything?

Answer. In simple linear regression $R^2 = r^2$, so $R^2 = 0.9782^2 \approx 0.957$.

Q5. Why do we square the residuals instead of adding them up?

Answer. Positive and negative residuals cancel in a plain sum, so even a poor line could total zero. Squaring makes every error count, and gives large errors extra weight, and it also gives a smooth bowl-shaped loss with a single minimum.

Q6. A model trained on 1 to 10 years of experience is asked to predict the salary for 25 years. Why is this a problem?

Answer. It is extrapolation. The data contain no evidence about 25 years, and the straight-line trend may bend or level off. Predictions are only reliable inside the range of X used for training.

Q7. Four datasets give the same line and the same r. Does this mean they are equally well described by a straight line?

Answer. No. Anscombe's quartet shows that identical summary numbers can hide a curve, an outlier, or a single influential point. Plot the data and the residuals.

Q8. What is the difference between simple and multiple linear regression, and which regression type would you try if two of your inputs were highly correlated?

Answer. Simple Linear Regression uses one input, Multiple Linear Regression uses two or more. With highly correlated inputs, the coefficients of ordinary Multiple Linear Regression become unstable, so Ridge Regression (or Elastic Net Regression) is a good option because the penalty shrinks and stabilises them.

Glossary

Term	Meaning
Regression	Predicting a numerical value from inputs
Simple Linear Regression (SLR)	A straight-line model with one independent variable
Multiple Linear Regression (MLR)	A linear model with two or more independent variables and one dependent variable
Polynomial Regression	Linear regression on powers of x (x, x^2, x^3 …); fits a curve but is still linear in the coefficients
Ridge Regression (L2)	Least squares with a penalty on the squared size of the coefficients
Lasso Regression (L1)	Least squares with a penalty on the absolute size of the coefficients; can set some to exactly 0
Elastic Net Regression	Least squares with a mix of the L1 and L2 penalties
Intercept (b_0, c)	The predicted y when x = 0
Slope (b_1, m, coefficient)	The predicted change in y for a one-unit increase in x
Residual	Actual y minus predicted $\hat{y}$ for one row
Least squares (OLS)	Choosing b_0 and b_1 that minimise the sum of squared residuals
SSE / SSR / SST	Unexplained / explained / total sum of squares; SST = SSR + SSE
R^2	Share of the variation in y explained by the model: 1 − SSE/SST
MAE / MSE / RMSE	Mean absolute error / mean squared error / its square root
Extrapolation	Predicting outside the range of X seen during training
Homoscedasticity	Residuals have the same spread for all x values
Outlier / influential point	A point far from the rest / one that changes the fitted line a lot
Standard error	How much an estimated quantity (such as b_1) would vary from sample to sample
Gradient descent	Minimising a loss by repeated small steps downhill

Sources

- scikit-learn user guide: *Linear Models* (ordinary least squares, ridge, lasso, elastic net), scikit-learn.org/stable/modules/linear_model.html
- statsmodels documentation: *Ordinary Least Squares* and `add_constant`, statsmodels.org
- Anscombe, F. J. (1973). *Graphs in Statistical Analysis*. The American Statistician, 27(1), 17-21.
- Class slides, *Understanding Linear Regression Algorithm* and *Mean Square Error* (the five-point example), and the Salary_Data.csv dataset used in class.
- Numbers in this document were recomputed with scikit-learn 1.9.1, statsmodels 0.15.0 and SciPy 1.18.1.